

Implementation Strategy for `library-port`

1. Summary

Yes. A better C++ implementation strategy than the current `library-port` is:

- template only the model-dependent kernels
- keep the solver/runtime classes non-templated
- compile the backend into a static core library
- generate or instantiate a small model-specific adapter TU
- isolate the legacy ABI path into a compatibility layer, not the main architecture

The core problem with current `library-port` is not the idea of templated models. It is that the template boundary is too wide: `M` propagates through too much of the backend.

So the better strategy is:

- narrow template boundary
- compiled core
- thin model bridge
- explicit instantiation where possible

2. Assumptions

- You are asking strictly about C++ implementation architecture, not whether the library workflow is useful.
- The current `library-port` strategy is the one where many backend classes are templated on `M`, including wrappers like `CDiscretization<M>`, and the external package is exposed through `Exasim::headers`.
- Exasim still needs:
 - compile-time access to model hooks inside Kokkos device code
 - CPU/MPI/GPU support
 - generated and handwritten models
- Inner-loop virtual dispatch is undesirable for performance and for device code.

3. Design reasoning

The current `library-port` implementation strategy does one important thing right:

- it makes model-specific pointwise math compile-time visible to Kokkos kernels

That part is sound.

What is weak is how far that decision spreads. Right now, the model type `M` influences a large portion of the backend object graph:

- discretization
- solution
- solver
- preconditioner
- point locator
- preprocessing-facing wrappers
- public include surface

That creates three maintainability problems.

A. Template contagion

Once `CDiscretization<M>` and `CSolution<M>` become the main organizing types, every helper touching them tends to become templated too. This causes:

- more headers
- more compile dependencies
- larger public surface
- more difficult refactors
- more confusing ownership boundaries

B. Too much backend becomes header code

The current strategy makes large backend subsystems effectively part of the user's compilation unit. That is not ideal for a large HPC solver. It raises:

- ODR risk
- build time
- compiler sensitivity
- packaging complexity
- transitive include fragility

C. The migration bridge leaks into the main design

`AbiAdapter` is a useful compatibility device, but it should be quarantined. In the current strategy, the legacy ABI story is still visible in the main dispatch architecture. That is acceptable during migration, but it is not the cleanest long-term implementation.

4. Proposed implementation

A better C++ strategy would be this.

A. Make the solver/runtime classes non-templated

Keep these non-templated or as close to non-templated as possible:

- solver/session orchestration
- mesh/preprocessing structures
- point locator
- time stepping
- I/O and postprocessing plumbing
- runtime ownership and memory management

In Exasim terms, most of:

- `backend/Discretization/`
- `backend/Solution/`
- `backend/Solver/`
- `backend/Preprocessing/`
- `backend/PointLocator/`

should not need to be templated all the way through.

Instead, these components should operate on runtime structs and call into model-dependent kernels through a narrow interface.

B. Template only the pointwise/kernel layer

Keep templates where they are actually justified:

- `include/exasim/model.hpp`
- `include/exasim/kernels/flux.hpp`
- `include/exasim/kernels/source.hpp`
- `include/exasim/kernels/boundary.hpp`

- `include/exasim/kernels/init.hpp`
- maybe QoI/output/vis kernels if they stay model-specific

This is the correct place for templates because:

- they need `M::flux`, `M::source`, etc.
- they need compile-time sizes like `M::nd`, `M::ncu`, `M::ncw`
- they run in Kokkos device code

C. Use a model adapter, not a templated solver hierarchy

Instead of templating the whole backend on `M`, create a small model-specific adapter layer, for example conceptually:

```
struct ModelOps {
    void (*flux)(...);
    void (*hdg_flux)(...);
    void (*source)(...);
    void (*hdg_source)(...);
    void (*fbou)(...);
    void (*fbou_hdg)(...);
    void (*initu)(...);
    ...
};
```

For a concrete model `MyModel`, generate or instantiate one adapter TU:

```
template <class M>
ModelOps make_model_ops();
```

That TU can call templated kernels internally, but the rest of the backend consumes only a runtime `ModelOps` or `ModelContext`.

This gives you:

- compile-time specialization where needed
- non-templated runtime orchestration everywhere else

D. Prefer explicit instantiation over transitive header expansion

For handwritten or generated models, a better strategy is:

- compile `my_model.hpp` into one dedicated `.cpp`
- explicitly instantiate the needed wrappers there
- link against `libexasim_core.a`

For example, conceptually:

```
#include "my_model.hpp"
#include <exasim/model_adapter.hpp>

template class exasim::ModelAdapter<MyModel>;

    or:

extern template class exasim::FluxKernel<MyModel>;
```

depending on how fine-grained you want instantiation.

This keeps the user model compile-time visible where needed without turning the whole backend into header-only infrastructure.

E. Use composition, not template inheritance cascades

Right now the branch tends toward “everything becomes `Thing<M>`”.

A better style is:

- `SolverCore` owns runtime state
- `DiscretizationCore` owns discretization state
- `ModelContext` supplies model-dependent operations
- `RunSession` manages MPI/Kokkos lifecycle
- `ExasimSolver` composes those pieces

That is easier to reason about than a deep template cascade of large classes.

F. Keep the ABI shim separate

The legacy ABI path should become one isolated adapter module, not part of the primary mental model.

That means:

- keep something like `AbiAdapter` only for old workflows
- do not let it dictate public API structure
- avoid `if constexpr` ABI-vs-template branching in large portions of the backend if you can centralize it earlier

5. Risks or numerical concerns

There are two important caveats.

- Do not move model-dependent HDG assembly behind virtual calls in the inner loop. That would hurt performance and complicate device execution. The better strategy is not runtime polymorphism inside kernels; it is runtime orchestration plus compile-time kernel specialization.

- Do not over-separate the model from the kernels. Functions like `flux`, `source`, `fbou_hdg`, and their Jacobians need to stay tightly coupled to the kernel code that gathers/scatters SoA data. If you move that boundary too far outward, you recreate the older ABI awkwardness.

So the best strategy is not:

- “object-oriented virtual solver everywhere”

It is:

- “compiled runtime core”
- “templated kernel edge”
- “small model adapter bridge”

6. Verification plan

If you wanted to test whether this strategy is actually better, I would prototype it with one case:

1. Build `libexasim_core.a` from the non-model-specific backend.
2. Keep only `flux`/`source`/`boundary`/`init` kernels templated.
3. Create one model adapter TU for `poisson2d`.
4. Link a small handwritten consumer against:
 - `libexasim_core.a`
 - the model adapter object
5. Compare against current `library-port` on:
 - compile time
 - include graph complexity
 - install/package simplicity
 - CPU/MPI/GPU behavior
 - numerical equivalence

So the short answer is:

- the better strategy is not “remove templates”
- it is not “keep templating the whole backend”
- it is template only the kernels, compile the rest, and bridge them with a small model-specific adapter